# Name: Hana Samy Ahmed

# Project 1: Spartan6 - DSP48A1

## 1. RTL code

```verilog
module BLOCK (D,SEL,CLK,RST,CE,BLOCK_OUT);
parameter RSTTYPE = "SYNC";
parameter WIDTH = 18;//No. of bus bits
input SEL,CLK,RST,CE;
input [WIDTH-1:0] D;
reg [WIDTH-1:0] Q;//Intetnal signal not an output
output  [WIDTH-1:0] BLOCK_OUT;
generate
    if (RSTTYPE == "SYNC") begin
        always @(posedge CLK) begin
            if (RST) begin
                Q<=0;
            end
            else if (CE) begin
                Q<=D;
            end
        end
    end
    else if (RSTTYPE == "ASYNC") begin
        always @(posedge CLK or posedge RST) begin
            if (RST) begin
                Q<=0;
            end
            else if (CE) begin
                Q<=D;
            end
        end
    end
endgenerate
assign BLOCK_OUT = (SEL)? Q : D ;
endmodule
```

```verilog
module DSP48A1 (A,B,D,C,CLK,CARRYIN,OPMODE,BCIN,RSTA,RSTB,RSTM,RSTP,RSTC,RSTD,RSTCARRYIN,RSTOPMODE,
                CEA,CEB,CEM,CEP,CEC,CED,CECARRYIN,CEOPMODE,PCIN,BCOUT,PCOUT,P,M,CARRYOUT,CARRYOUTF);
//Parameter (Attributes):
parameter A0REG = 0 ;//0 : No Register
parameter A1REG = 1 ;//1 : Registerd
parameter B0REG = 0 ;
parameter B1REG = 1 ;
parameter CREG = 1;
parameter DREG = 1 ;
parameter MREG = 1 ;
parameter PREG = 1 ;
parameter CARRYINREG = 1 ;
parameter CARRYOUTREG = 1 ;
parameter OPMODEREG = 1;
parameter CARRYINSEL = "OPMODE5" ;// CARRYIN or OPMODE5 , Default: OPMODE5
parameter B_INPUT = "DIRECT" ; // DIRECT from B input or CASCADE from the previous DSP48A1 slice , Default: DIRECT, if neither MUX out =0
parameter RSTTYPE = "SYNC" ; //ASYNC or SYNC , Default: SYNC
//Data Ports:
input [17:0] A,B,D;
input [47:0] C;
output [35:0] M;
output [47:0] P;
input CARRYIN ;
output CARRYOUT, CARRYOUTF;
//Control Input Ports:
input CLK;
input [7:0] OPMODE;
//Clock Enable Input Ports:
input CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP;
//Reset Input Ports:
input RSTA, RSTB, RSTC, RSTCARRYIN, RSTD, RSTM, RSTOPMODE, RSTP;
//Cascade Ports:
output [17:0] BCOUT;
input [47:0] PCIN ;
output [47:0] PCOUT ;
input [17:0] BCIN ;
```

```verilog
//Design
//First Level
wire [17:0] DREGOUT ;
BLOCK #(.WIDTH(18),.RSTTYPE(RSTTYPE)) DMUXREG (.D(D),.SEL(DREG),.CLK(CLK),.RST(RSTD),.CE(CED),.BLOCK_OUT(DREGOUT));

wire [17:0] B0MUXOUT ;
wire [17:0] B0REGOUT ;
assign B0MUXOUT = (B_INPUT == "DIRECT") ? B : (B_INPUT == "CASCADE")? BCIN : 0 ;
BLOCK #(.WIDTH(18),.RSTTYPE(RSTTYPE)) B0MUXREG (.D(B0MUXOUT),.SEL(B0REG),.CLK(CLK),.RST(RSTB),.CE(CEB),.BLOCK_OUT(B0REGOUT));

wire [17:0] A0REGOUT ;
BLOCK #(.WIDTH(18),.RSTTYPE(RSTTYPE)) A0MUXREG (.D(A),.SEL(A0REG),.CLK(CLK),.RST(RSTA),.CE(CEA),.BLOCK_OUT(A0REGOUT));

wire [47:0] CREGOUT ;
BLOCK #(.WIDTH(48),.RSTTYPE(RSTTYPE)) CMUXREG (.D(C),.SEL(CREG),.CLK(CLK),.RST(RSTC),.CE(CEC),.BLOCK_OUT(CREGOUT));

//Second Level
wire [7:0] OPMODEREGOUT ;
BLOCK #(.WIDTH(8),.RSTTYPE(RSTTYPE)) OPMODEMUXREG (.D(OPMODE),.SEL(OPMODEREG),.CLK(CLK),.RST(RSTOPMODE),.CE(CEOPMODE),.BLOCK_OUT(OPMODEREGOUT));

wire [17:0] PREADDEROUT;
assign PREADDEROUT = (OPMODEREGOUT[6]) ? (DREGOUT - B0REGOUT) : (DREGOUT + B0REGOUT) ;

wire [17:0] PREMUXOUT ;
assign PREMUXOUT = (OPMODEREGOUT[4]) ? PREADDEROUT : B0REGOUT ;

//Third Level
wire [17:0] B1REGOUT ;
BLOCK #(.WIDTH(18),.RSTTYPE(RSTTYPE)) B1MUXREG (.D(PREMUXOUT),.SEL(B1REG),.CLK(CLK),.RST(RSTB),.CE(CEB),.BLOCK_OUT(B1REGOUT));

wire [17:0] A1REGOUT ;
BLOCK #(.WIDTH(18),.RSTTYPE(RSTTYPE)) A1MUXREG (.D(A0REGOUT),.SEL(A1REG),.CLK(CLK),.RST(RSTA),.CE(CEA),.BLOCK_OUT(A1REGOUT));
```

```verilog
//Fourth Level
assign BCOUT = B1REGOUT ;

wire [35:0] MULTIPLIEROUT ;
assign MULTIPLIEROUT = B1REGOUT * A1REGOUT ;

wire [35:0] MREGOUT;
BLOCK #(.WIDTH(36),.RSTTYPE(RSTTYPE)) MMUXREG (.D(MULTIPLIEROUT),.SEL(MREG),.CLK(CLK),.RST(RSTM),.CE(CEM),.BLOCK_OUT(MREGOUT));

generate
    genvar i;
    for(i = 0;i<36;i=i+1)
        buf(M[i],MREGOUT[i]);
endgenerate

wire CARRYMUXOUT ;
assign CARRYMUXOUT = (CARRYINSEL == "OPMODE5") ? OPMODEREGOUT[5] : (CARRYINSEL == "CARRYIN") ? CARRYIN : 0 ;

wire CIN;
BLOCK #(.WIDTH(1),.RSTTYPE(RSTTYPE)) CYIMUXREG (.D(CARRYMUXOUT),.SEL(CARRYINREG),.CLK(CLK),.RST(RSTCARRYIN),.CE(CECARRYIN),.BLOCK_OUT(CIN));

//Fifth Level
wire [47:0] MUXXOUT;
wire [47:0] DABCONCAT ;
assign DABCONCAT = { D[11:0], A[17:0], B[17:0]} ;
assign MUXXOUT = (OPMODEREGOUT[1:0] ==0 ) ? 0 : (OPMODEREGOUT[1:0] ==1) ? MREGOUT : (OPMODEREGOUT[1:0] ==2) ? P : DABCONCAT ;

wire [47:0] MUXZOUT;
assign MUXZOUT = (OPMODEREGOUT[3:2] ==0 ) ? 0 : (OPMODEREGOUT[3:2] ==1) ? PCIN : (OPMODEREGOUT[3:2] ==2) ? P : CREGOUT ;

wire [47:0] POSTADDERSUM;
wire POSTADDERCOUT;
assign {POSTADDERCOUT,POSTADDERSUM} = (OPMODEREGOUT[7]) ? (MUXZOUT - (MUXXOUT + CIN)) : (MUXZOUT + MUXXOUT + CIN)  ;
```

```verilog

BLOCK #(.WIDTH(1),.RSTTYPE(RSTTYPE)) CYOMUXREG (.D(POSTADDERCOUT),.SEL(CARRYOUTREG),.CLK(CLK),.RST(RSTCARRYIN),.CE(CECARRYIN),.BLOCK_OUT(CARRYOUT
assign CARRYOUTF = CARRYOUT;

BLOCK #(.WIDTH(48),.RSTTYPE(RSTTYPE)) PMUXREG (.D(POSTADDERSUM),.SEL(PREG),.CLK(CLK),.RST(RSTP),.CE(CEP),.BLOCK_OUT(P));
assign PCOUT = P ;
endmodule
```

# 2. Testbench code

```verilog
`timescale 1ns / 1ps

module tb_DSP48A1;

    // ----------------------------------------------------------
    // Inputs
    // ----------------------------------------------------------
    reg [17:0] A, B, D;
    reg [47:0] C;
    reg CLK;
    reg CARRYIN;
    reg [7:0] OPMODE;
    reg [17:0] BCIN;
    reg [47:0] PCIN;

    // Resets
    reg RSTA, RSTB, RSTM, RSTP, RSTC, RSTD, RSTCARRYIN, RSTOPMODE;

    // Clock Enables
    reg CEA, CEB, CEM, CEP, CEC, CED, CECARRYIN, CEOPMODE;

    // ----------------------------------------------------------
    // Outputs
    // ----------------------------------------------------------
    wire [35:0] M;
    wire [47:0] P;
    wire CARRYOUT, CARRYOUTF;
    wire [17:0] BCOUT;
    wire [47:0] PCOUT;

    // Internal variables for checking
    integer errors = 0;
    reg [47:0] past_P;
    reg past_CARRYOUT;

```

```verilog
    // -----------------------------------------------------------
    // Instantiate the Unit Under Test (UUT)
    // -----------------------------------------------------------
    DSP48A1 uut (
        .A(A), .B(B), .D(D), .C(C), .CLK(CLK), .CARRYIN(CARRYIN),
        .OPMODE(OPMODE), .BCIN(BCIN),
        .RSTA(RSTA), .RSTB(RSTB), .RSTM(RSTM), .RSTP(RSTP),
        .RSTC(RSTC), .RSTD(RSTD), .RSTCARRYIN(RSTCARRYIN), .RSTOPMODE(RSTOPMODE),
        .CEA(CEA), .CEB(CEB), .CEM(CEM), .CEP(CEP),
        .CEC(CEC), .CED(CED), .CECARRYIN(CECARRYIN), .CEOPMODE(CEOPMODE),
        .PCIN(PCIN), .BCOUT(BCOUT), .PCOUT(PCOUT), .P(P), .M(M),
        .CARRYOUT(CARRYOUT), .CARRYOUTF(CARRYOUTF)
    );

    // -----------------------------------------------------------
    // Clock Generation
    // -----------------------------------------------------------
    initial begin
        CLK = 0;
        forever #5 CLK = ~CLK; // 10ns period
    end

    // -----------------------------------------------------------
    // Verification Sequence
    // -----------------------------------------------------------
    initial begin
        $display("=================================================");
        $display("       STARTING DSP48A1 VERIFICATION SEQUENCE    ");
        $display("=================================================");

        // =======================================================
        // 2.1. Verify Reset Operation
        // =======================================================
```

```
$display("\n---> Running Test 2.1: Verify Reset Operation");
// Assert all active-high reset signals
RSTA = 1; RSTB = 1; RSTM = 1; RSTP = 1;
RSTC = 1; RSTD = 1; RSTCARRYIN = 1; RSTOPMODE = 1;
// Drive remaining inputs with arbitrary values
A = $random; B = $random; C = $random; D = $random;
CARRYIN = $random; OPMODE = $random; BCIN = $random; PCIN = $random;

// Wait for the negative edge of the clock
@(negedge CLK);

// Self-checking condition
if (BCOUT !== 0 || M !== 0 || P !== 0 || PCOUT !== 0 || CARRYOUT !== 0 || CARRYOUTF !== 0) begin
    $display("[FAIL] 2.1 Reset failed. Outputs are not zero.");
    errors = errors + 1;
end else begin
    $display("[PASS] 2.1 Reset successful. All outputs are zero.");
end

// Deassert resets and assert clock enables
RSTA = 0; RSTB = 0; RSTM = 0; RSTP = 0;
RSTC = 0; RSTD = 0; RSTCARRYIN = 0; RSTOPMODE = 0;
CEA = 1; CEB = 1; CEM = 1; CEP = 1;
CEC = 1; CED = 1; CECARRYIN = 1; CEOPMODE = 1;


// =======================================================
// 2.2. Verify DSP Path 1
// =======================================================
$display("\n---> Running Test 2.2: Verify DSP Path 1");
// ALREADY AT NEGEDGE: Apply inputs immediately
OPMODE = 8'b11011101;
A = 20; B = 10; C = 350; D = 25;
BCIN = $random; PCIN = $random; CARRYIN = $random;

// Wait for four negative clock edges
```

```
    repeat(4) @(negedge CLK);

    // Self-checking condition
    if (BCOUT !== 18'hf || M !== 36'h12c || P !== 48'h32 || PCOUT !== 48'h32 || CARRYOUT !== 1'b0 || CARRYOUTF !== 1'b0) begin
        $display("[FAIL] 2.2 Path 1 failed.");
        $display("       Got BCOUT=%h, M=%h, P=%h, CARRYOUT=%b", BCOUT, M, P, CARRYOUT);
        errors = errors + 1;
    end else begin
        $display("[PASS] 2.2 Path 1 successful.");
    end


    // =============================================================
    // 2.3. Verify DSP Path 2
    // =============================================================
    $display("\n---> Running Test 2.3: Verify DSP Path 2");
    OPMODE = 8'b00010000;
    A = 20; B = 10; C = 350; D = 25;
    BCIN = $random; PCIN = $random; CARRYIN = $random;

    // Wait for three negative edges
    repeat(3) @(negedge CLK);

    // Self-checking condition
    if (BCOUT !== 18'h23 || M !== 36'h2bc || P !== 48'h0 || PCOUT !== 48'h0 || CARRYOUT !== 1'b0 || CARRYOUTF !== 1'b0) begin
        $display("[FAIL] 2.3 Path 2 failed.");
        $display("       Got BCOUT=%h, M=%h, P=%h, CARRYOUT=%b", BCOUT, M, P, CARRYOUT);
        errors = errors + 1;
    end else begin
        $display("[PASS] 2.3 Path 2 successful.");
    end

    // Store past values for Path 3 check
    past_P = P;
    past_CARRYOUT = CARRYOUT;
```

```
    // =============================================================
    // 2.4. Verify DSP Path 3
    // =============================================================
    $display("\n---> Running Test 2.4: Verify DSP Path 3");
    OPMODE = 8'b00001010;
    A = 20; B = 10; C = 350; D = 25;
    BCIN = $random; PCIN = $random; CARRYIN = $random;

    // Wait for three negative edges
    repeat(3) @(negedge CLK);

    // Self-checking condition
    if (BCOUT !== 18'ha || M !== 36'hc8 || P !== past_P || PCOUT !== past_P || CARRYOUT !== past_CARRYOUT) begin
        $display("[FAIL] 2.4 Path 3 failed.");
        $display("       Got BCOUT=%h, M=%h, P=%h, CARRYOUT=%b", BCOUT, M, P, CARRYOUT);
        errors = errors + 1;
    end else begin
        $display("[PASS] 2.4 Path 3 successful.");
    end


    // =============================================================
    // 2.5. Verify DSP Path 4
    // =============================================================
    $display("\n---> Running Test 2.5: Verify DSP Path 4");
    OPMODE = 8'b10100111;
    A = 5; B = 6; C = 350; D = 25; PCIN = 3000;
    BCIN = $random; CARRYIN = $random;

    // Wait for three negative edges
    repeat(3) @(negedge CLK);
```

```verilog
    // Self-checking condition
    if (BCOUT !== 18'h6 || M !== 36'h1e || P !== 48'hfe6fffec0bb1 || PCOUT !== 48'hfe6fffec0bb1 || CARRYOUT !== 1'b1 || CARRYOUTF !== 1'b1) begin
        $display("[FAIL] 2.5 Path 4 failed.");
        $display("      Got BCOUT=%h, M=%h, P=%h, CARRYOUT=%b", BCOUT, M, P, CARRYOUT);
        errors = errors + 1;
    end else begin
        $display("[PASS] 2.5 Path 4 successful.");
    end

    // ------------------------------------------------------------
    // End of Simulation Summary
    // ------------------------------------------------------------
    $display("\n------------------------------------------------------");
    if (errors == 0) begin
        $display("   [SUCCESS] ALL VERIFICATION SCENARIOS PASSED!   ");
    end else begin
        $display("   [WARNING] SIMULATION FINISHED WITH %0d ERRORS.", errors);
    end
    $display("------------------------------------------------------");

    #20;
    $finish;
  end

endmodule
```

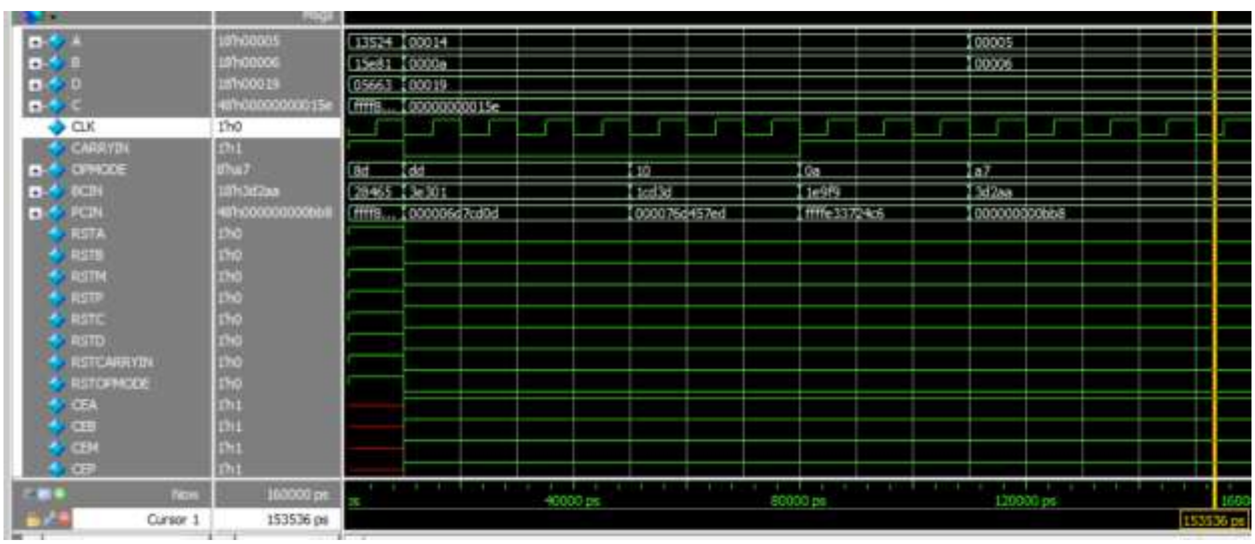
# 3. Do file

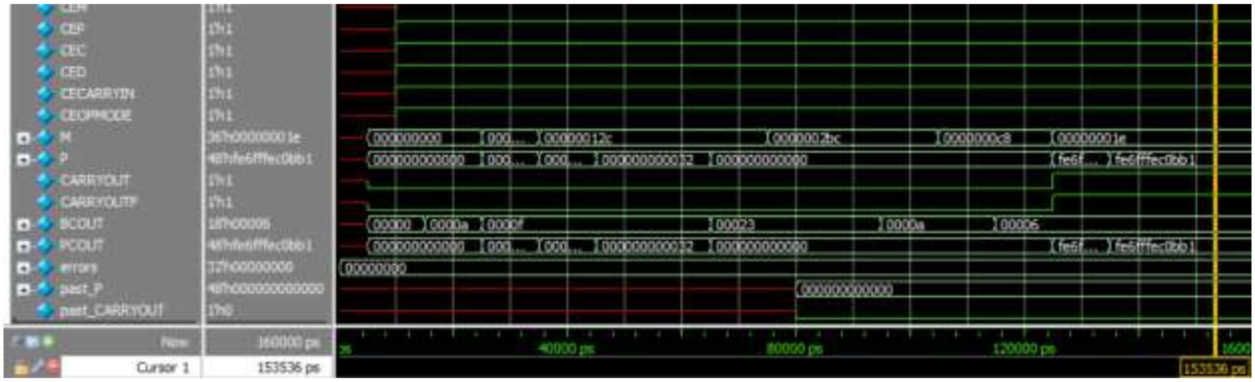
DSP48A1.do - Notepad

File Edit Format View Help

```
vlib work
vlog BLOCK.V DSP48A1.v tb_DSP.v
vsim -voptargs=+acc work.tb_DSP48A1
add wave *
run -all
#quit -sim
```

# 4. QuestaSim Snippets






# 5. Transcript

```
# ==================================================
#        STARTING DSP48A1 VERIFICATION SEQUENCE
# ==================================================
#
# ---> Running Test 2.1: Verify Reset Operation
# [PASS] 2.1 Reset successful. All outputs are zero.
#
# ---> Running Test 2.2: Verify DSP Path 1
# [PASS] 2.2 Path 1 successful.
#
# ---> Running Test 2.3: Verify DSP Path 2
# [PASS] 2.3 Path 2 successful.
#
# ---> Running Test 2.4: Verify DSP Path 3
# [PASS] 2.4 Path 3 successful.
#
# ---> Running Test 2.5: Verify DSP Path 4
# [PASS] 2.5 Path 4 successful.
#
# ==================================================
#    [SUCCESS] ALL VERIFICATION SCENARIOS PASSED!
# ==================================================
```

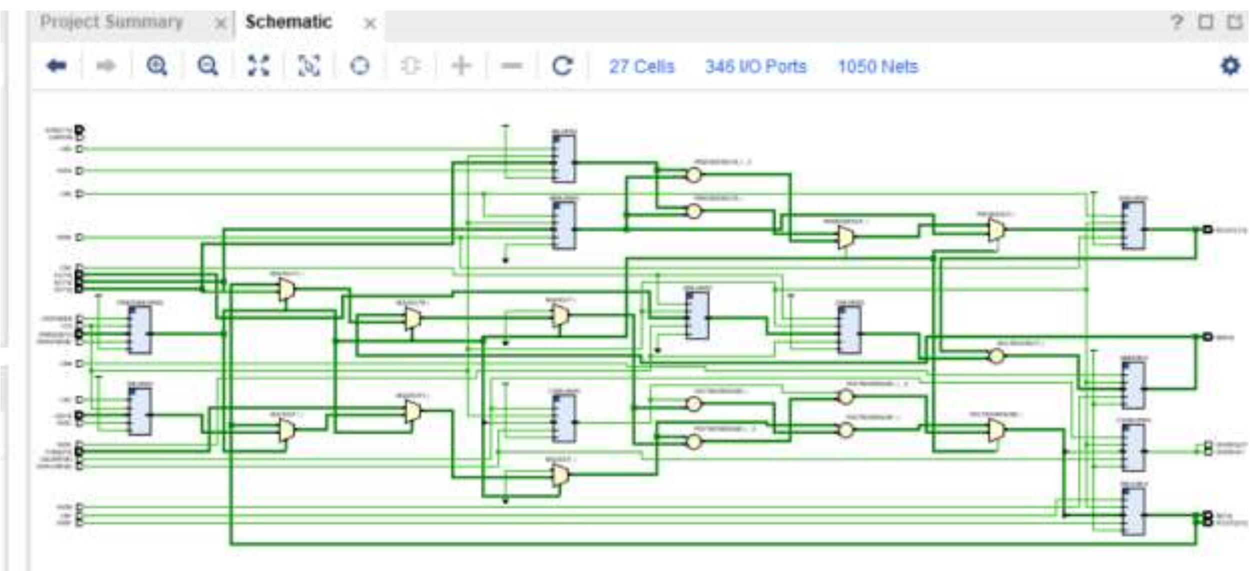
# 5. Constraint File

Constraints_basys3.xdc - Notepad

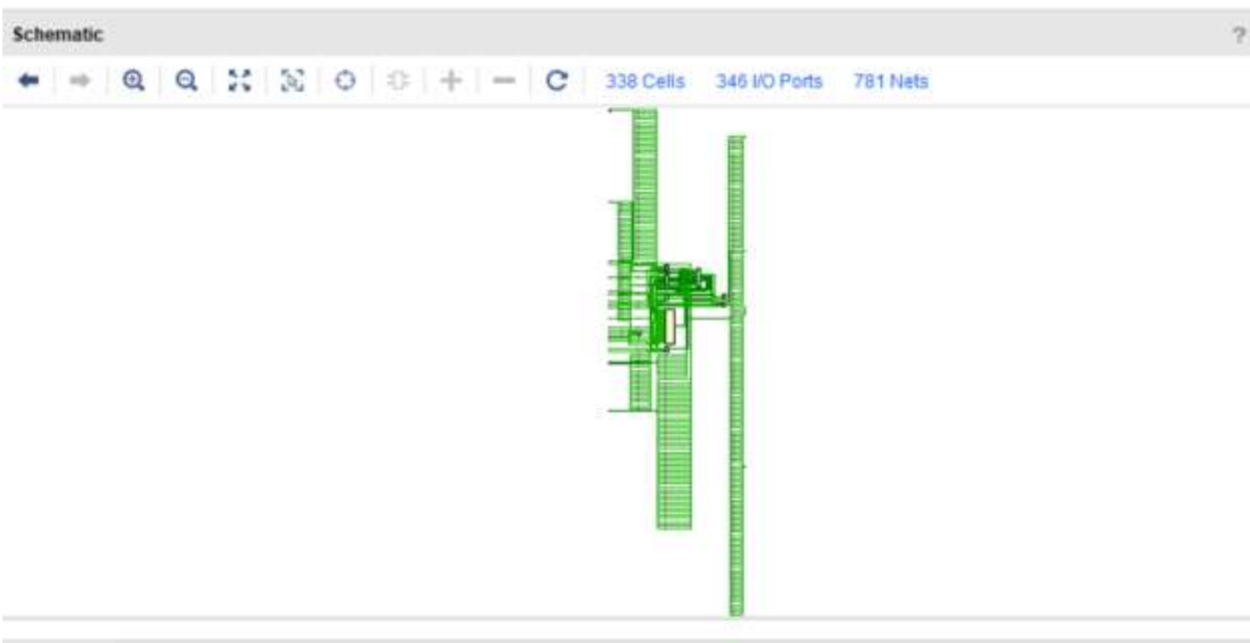
File Edit Format View Help

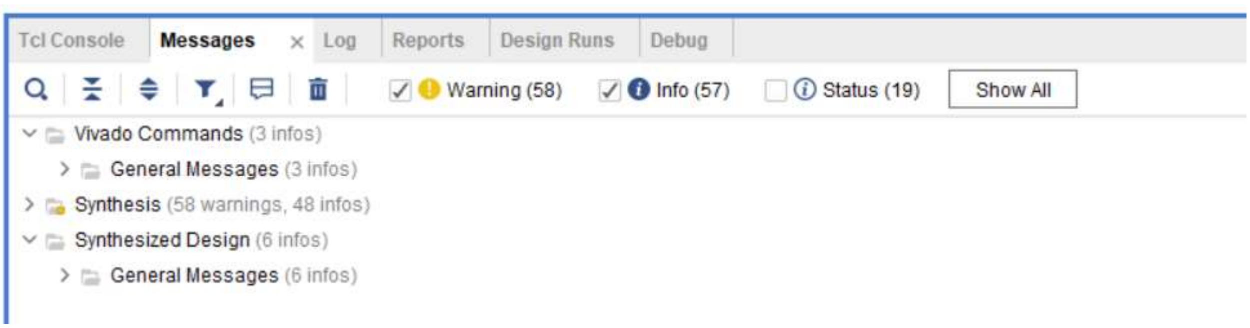
```
## This file is a general .xdc for the Basys3 rev B board
## To use it in a project:
## - uncomment the lines corresponding to used pins
## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project

## Clock signal
set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33} [get_ports CLK]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports CLK]


## Switches
#set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports {sw[0]}]
#set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports {sw[1]}]
#set_property -dict { PACKAGE_PIN W16   IOSTANDARD LVCMOS33 } [get_ports {sw[2]}]
#set_property -dict { PACKAGE_PIN W17   IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]
#set_property -dict { PACKAGE_PIN W15   IOSTANDARD LVCMOS33 } [get_ports {sw[4]}]
#set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports {sw[5]}]
#set_property -dict { PACKAGE_PIN W14   IOSTANDARD LVCMOS33 } [get_ports {sw[6]}]
#set_property -dict { PACKAGE_PIN W13   IOSTANDARD LVCMOS33 } [get_ports {sw[7]}]
#set_property -dict { PACKAGE_PIN V2    IOSTANDARD LVCMOS33 } [get_ports {sw[8]}]
#set_property -dict { PACKAGE_PIN T3    IOSTANDARD LVCMOS33 } [get_ports {sw[9]}]
#set_property -dict { PACKAGE_PIN T2    IOSTANDARD LVCMOS33 } [get_ports {sw[10]}]
#set_property -dict { PACKAGE_PIN R3    IOSTANDARD LVCMOS33 } [get_ports {sw[11]}]
#set_property -dict { PACKAGE_PIN W2    IOSTANDARD LVCMOS33 } [get_ports {sw[12]}]
#set_property -dict { PACKAGE_PIN U1    IOSTANDARD LVCMOS33 } [get_ports {sw[13]}]
#set_property -dict { PACKAGE_PIN T1    IOSTANDARD LVCMOS33 } [get_ports {sw[14]}]
#set_property -dict { PACKAGE_PIN R2    IOSTANDARD LVCMOS33 } [get_ports {sw[15]}]
```

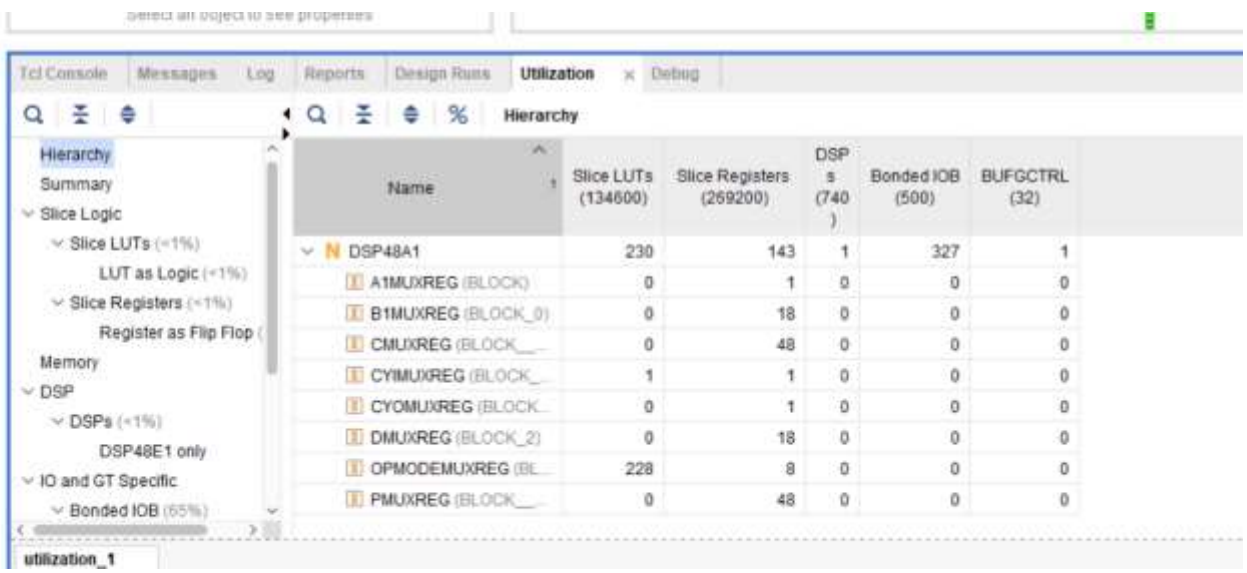
# 6. Elaboration ("Messages" tab & Schematic snippets)




7. Synthesis ("Messages" tab, Utilization report, timing report & Schematic snippets)

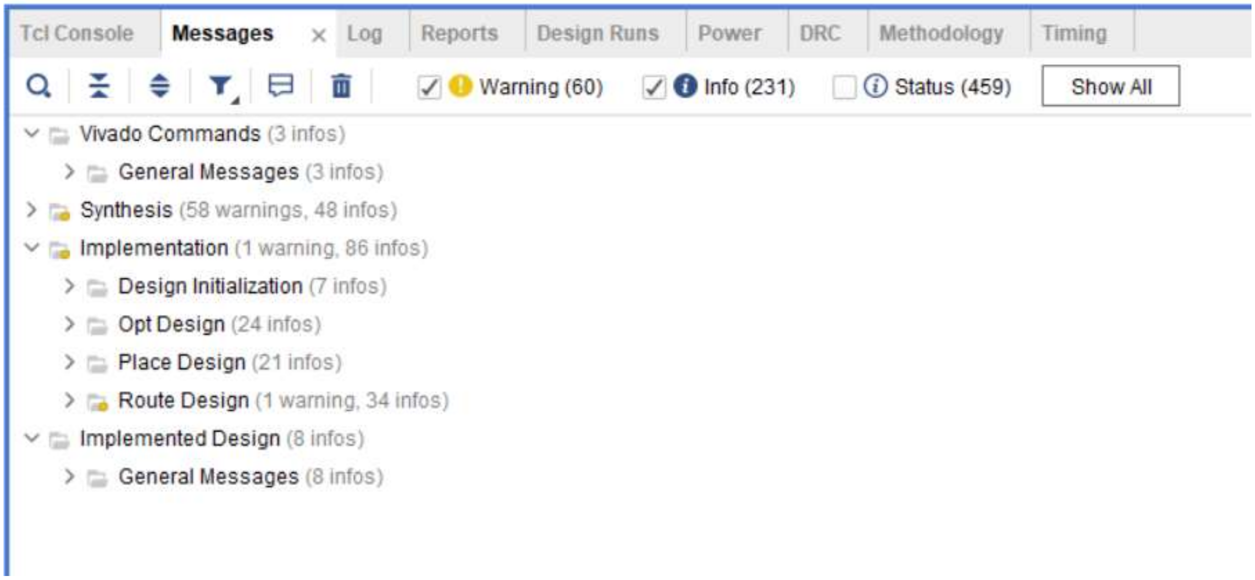


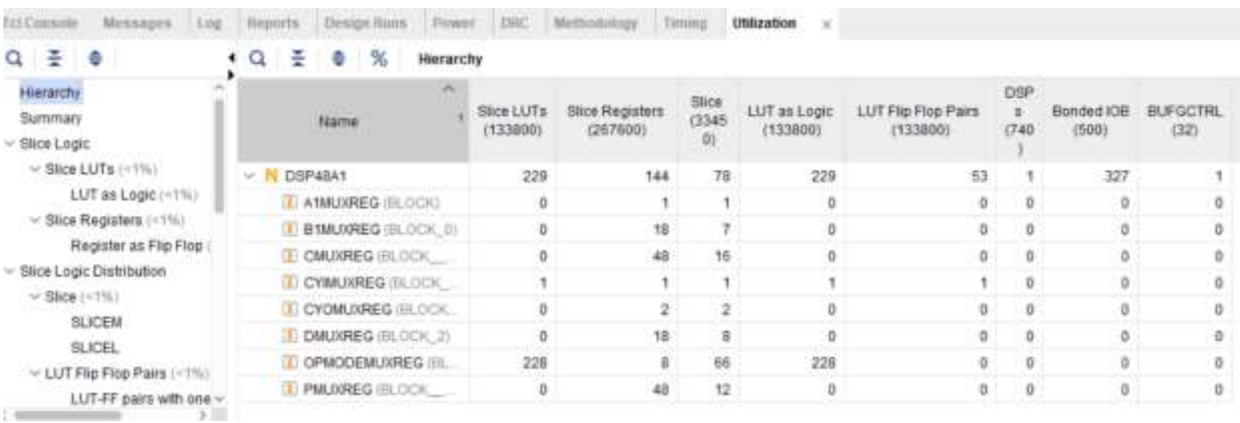





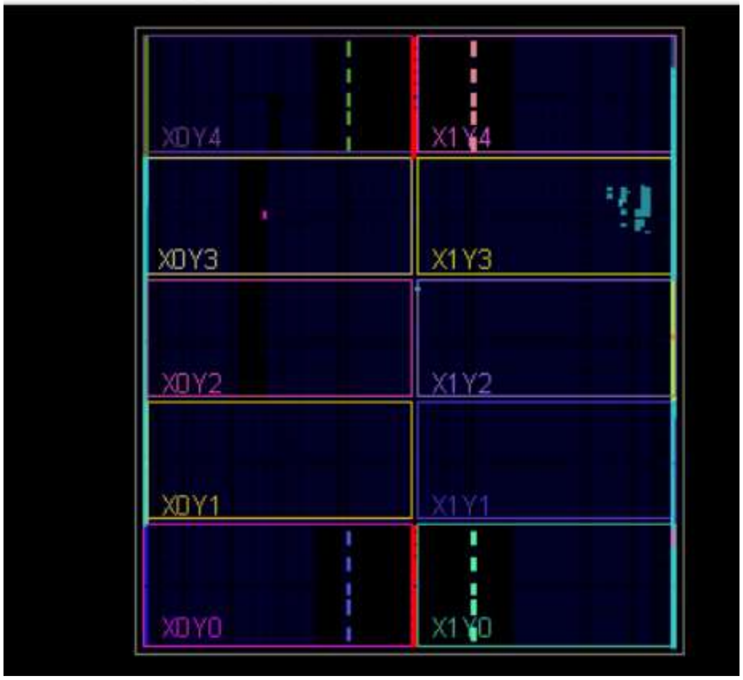


8. Implementation ("Messages" tab, Utilization report, timing report & device snippets)

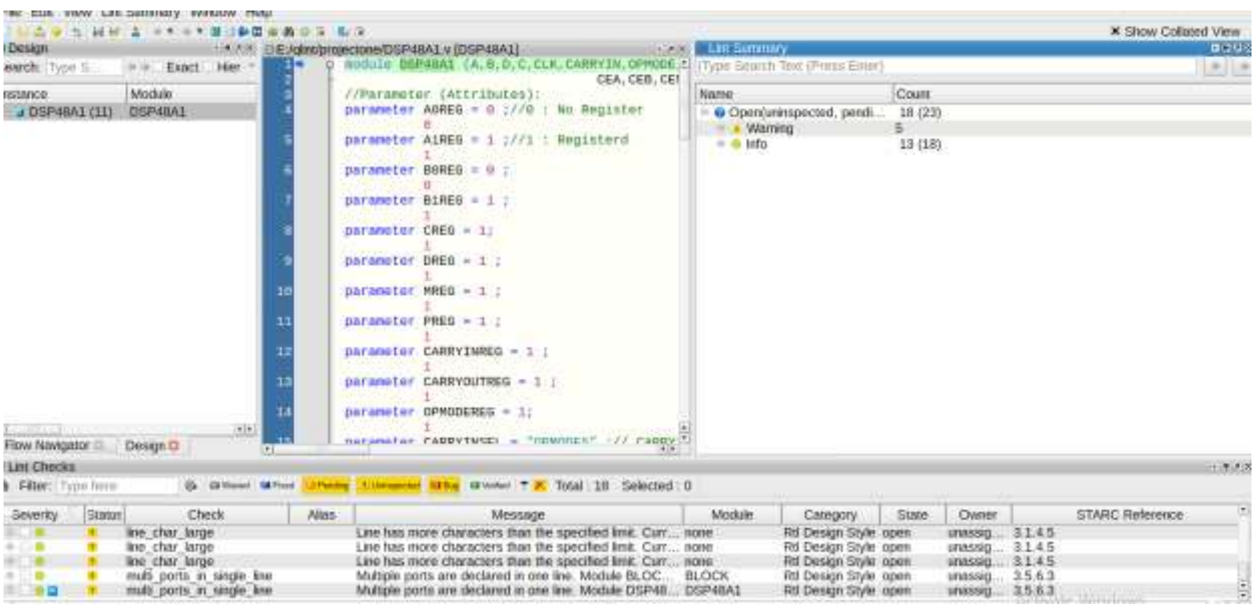

Tcl Console
Messages
Log
Reports
Design Runs
Power
DRC
Methodology
Timing
Warning (60)
Info (231)
Status (459)
Show All
Vivado Commands (3 infos)
General Messages (3 infos)
Synthesis (58 warnings, 48 infos)
Implementation (1 warning, 86 infos)
Design Initialization (7 infos)
Opt Design (24 infos)
Place Design (21 infos)
Route Design (1 warning, 34 infos)
Implemented Design (8 infos)
General Messages (8 infos)


Utilization
Hierarchy
Summary
Slice Logic
Slice LUTs (<1%)
LUT as Logic (<1%)
Slice Registers (<1%)
Register as Flip Flop
Slice Logic Distribution
Slice (<1%)
SLICEM
SLICEL
LUT Flip Flop Pairs (<1%)
LUT-FF pairs with one
Name | Slice LUTs (133800) | Slice Registers (267600) | Slice (33450) | LUT as Logic (133800) | LUT Flip Flop Pairs (133800) | DSPs (740) | Bonded IOB (500) | BUFGCTRL (32)
DSP48A1 | 229 | 144 | 78 | 229 | 53 | 1 | 327 | 1
A1MUXREG (BLOCK) | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0
B1MUXREG (BLOCK_0) | 0 | 18 | 7 | 0 | 0 | 0 | 0 | 0
CMUXREG (BLOCK_ | 0 | 48 | 16 | 0 | 0 | 0 | 0 | 0
CYIMUXREG (BLOCK_ | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0
CYOMUXREG (BLOCK | 0 | 2 | 2 | 0 | 0 | 0 | 0 | 0
DMUXREG (BLOCK_2) | 0 | 18 | 8 | 0 | 0 | 0 | 0 | 0
OPMODEMUXREG (BL | 228 | 8 | 66 | 228 | 0 | 0 | 0 | 0
PMUXREG (BLOCK_ | 0 | 48 | 12 | 0 | 0 | 0 | 0 | 0


Timing
Design Timing Summary
General Information
Timer Settings
Design Timing Summary
Clock Summary (1)
Check Timing (326)
Intra-Clock Paths
Inter-Clock Paths
Setup
Worst Negative Slack (WNS): 3.862 ns
Total Negative Slack (TNS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 107
Hold
Worst Hold Slack (WHS): 0.273 ns
Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 107
Pulse Width
Worst Pulse Width Slack (WPWS): 4.500 ns
Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 146
All user specified timing constraints are met.
Timing Summary - impl_1 (saved)




# 9. Linting (snippets showing no errors)



| Severity | Status | Check | Alias | Message | Module | Category | State | Owner | STARC Refere |
|---|---|---|---|---|---|---|---|---|---|
| | | mux_select_const | | Constant value drives mux select pin. Signal CMUXRE... | BLOCK | Connectivity | open | unassig... | |
| | | mux_select_const | | Constant value drives mux select pin. Signal CYIMUX... | BLOCK | Connectivity | open | unassig... | |
| | | mux_select_const | | Constant value drives mux select pin. Signal DMUXRE... | BLOCK | Connectivity | open | unassig... | |
| | | mux_select_const | | Constant value drives mux select pin. Signal MMUXR... | BLOCK | Connectivity | open | unassig... | |
| | | mux_select_const | | Constant value drives mux select pin. Signal OPMOD... | BLOCK | Connectivity | open | unassig... | |
| | | condition_const | | Condition expression is a constant. Module DSP48A1,... | DSP48A1 | Rtl Design Style | open | unassig... | |
| | | condition_const | | Condition expression is a constant. Module DSP48A1,... | DSP48A1 | Rtl Design Style | open | unassig... | |
| | | parameter_name_duplicate | | Same parameter name is used in more than one mod... | BLOCK | Nomenclature... | open | unassig... | 1.1.4.3, 3.2.2.2 |
| | | line_char_large | | Line has more characters than the specified limit. Curr... | none | Rtl Design Style | open | unassig... | 3.1.4.5 |
| | | line_char_large | | Line has more characters than the specified limit. Curr... | none | Rtl Design Style | open | unassig... | 3.1.4.5 |
| | | line_char_large | | Line has more characters than the specified limit. Curr... | none | Rtl Design Style | open | unassig... | 3.1.4.5 |
| | | line_char_large | | Line has more characters than the specified limit. Curr... | none | Rtl Design Style | open | unassig... | 3.1.4.5 |
| | | line_char_large | | Line has more characters than the specified limit. Curr... | none | Rtl Design Style | open | unassig... | 3.1.4.5 |
| | | line_char_large | | Line has more characters than the specified limit. Curr... | none | Rtl Design Style | open | unassig... | 3.1.4.5 |
| | | line_char_large | | Line has more characters than the specified limit. Curr... | none | Rtl Design Style | open | unassig... | 3.1.4.5 |
| | | line_char_large | | Line has more characters than the specified limit. Curr... | none | Rtl Design Style | open | unassig... | 3.1.4.5 |
| | | multi_ports_in_single_line | | Multiple ports are declared in one line. Module BLOC... | BLOCK | Rtl Design Style | open | unassig... | 3.5.6.3 |
| | | multi_ports_in_single_line | | Multiple ports are declared in one line. Module DSP48... | DSP48A1 | Rtl Design Style | open | unassig... | 3.5.6.3 |